CS360 Introduction to Database

Secondary Storage Management

Min-Soo Kim

School of Computing, KAIST

Class schedule

- Chapter 13. Second storage management
- Chapter 14. Index structures
- Chapter 15. Query execution
- Chapter 16. Query optimization

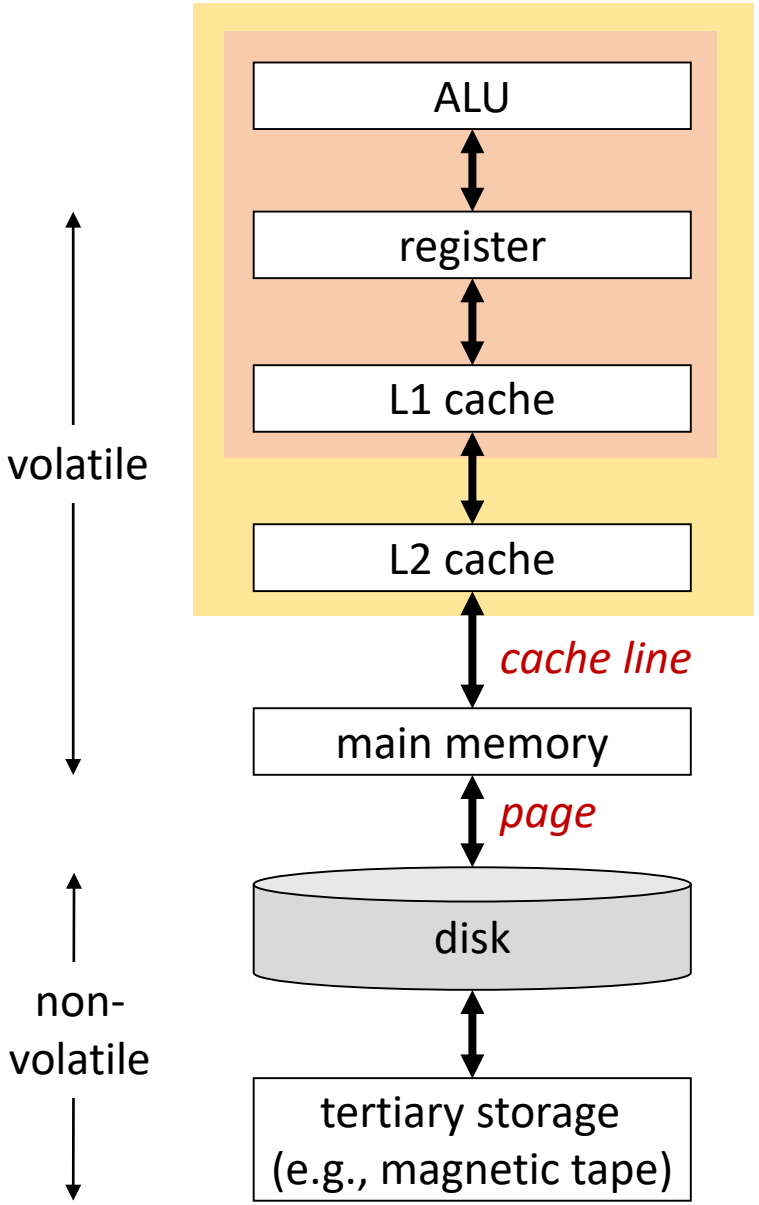
Scope of the final exam

- Chapter 8. Views and indexes
- Chapter 10. On-line analytic processing

Outline

- Memory hierarchy
- Disk access time
- Double buffering technique
- RAID (Redundant Array of Independent Disks)
- Database address space
- Pointer swizzling
- Record formats
- Record modification

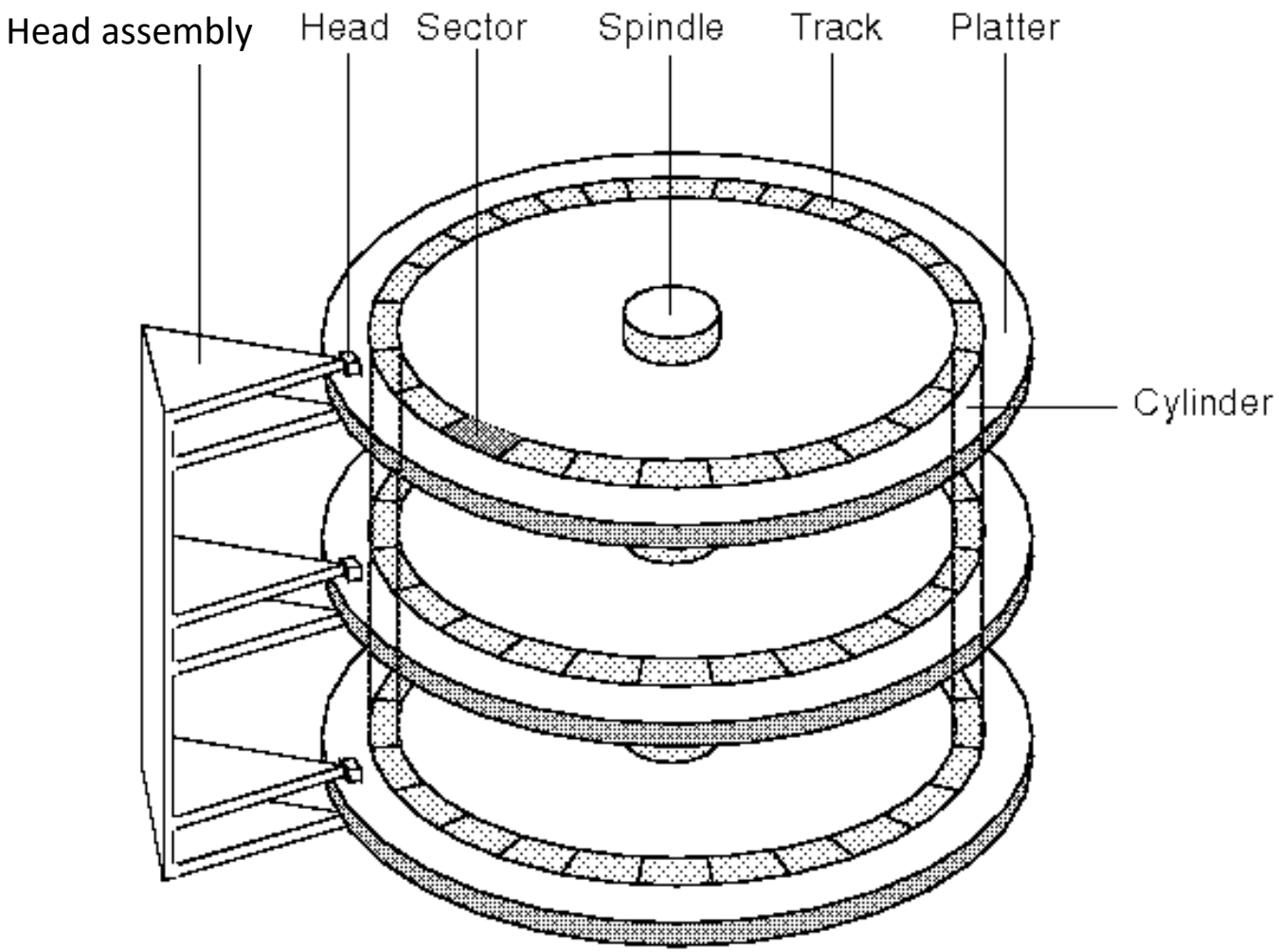
Memory hierarchy



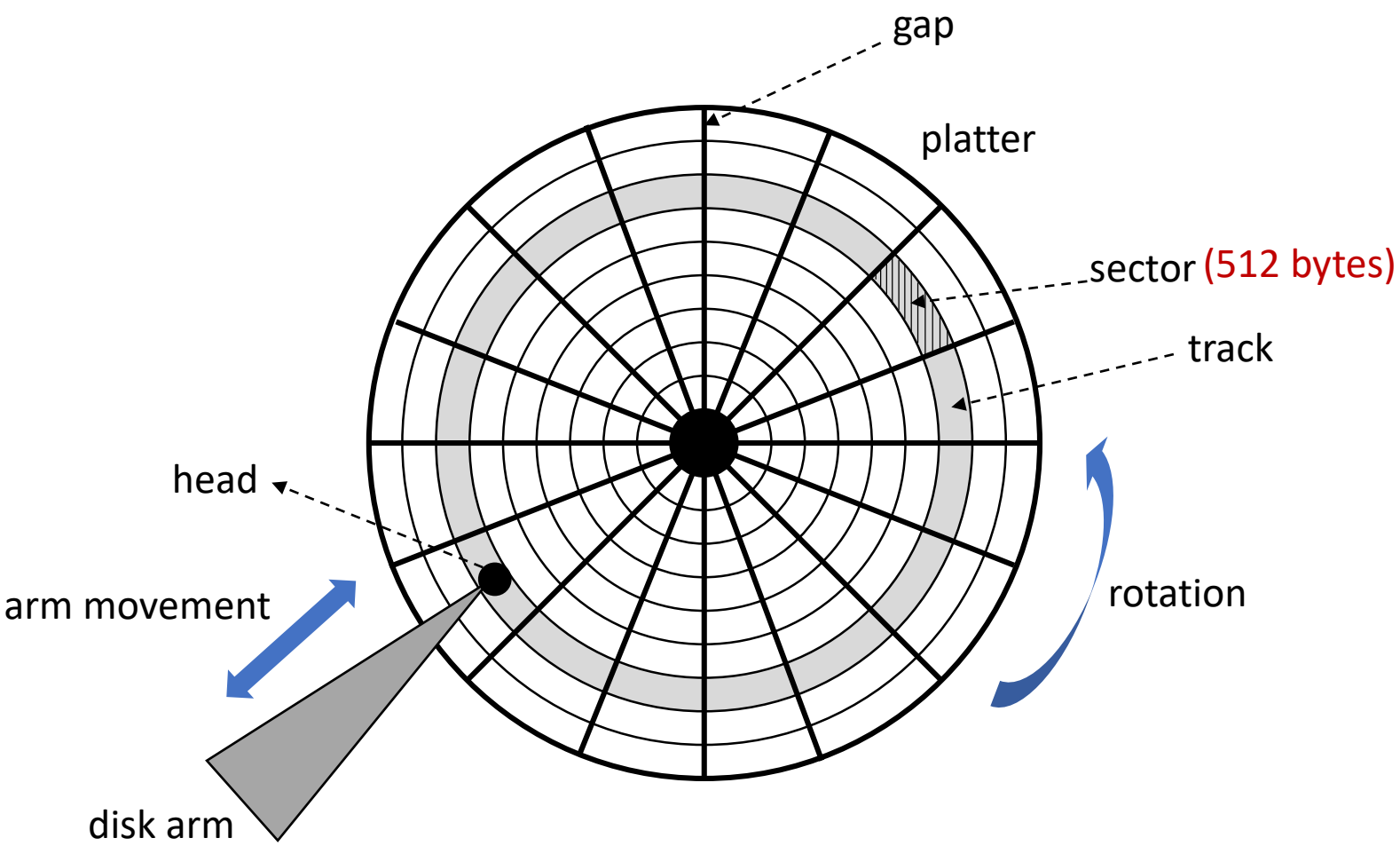
	Speed	Capacity
ALU	0.3 nanosec	
register	0.5 nanosec	64bit / 512bit
L1 cache	7 nanosec	256KB ~ a few MB
L2 cache	~100 nanosec	a few ~ several tens MB
main memory	~10,000,000 nanosec (seek)	~ a few TB
disk		~ several tens TB

Disks

7200 RPM (Revolutions Per Minute)

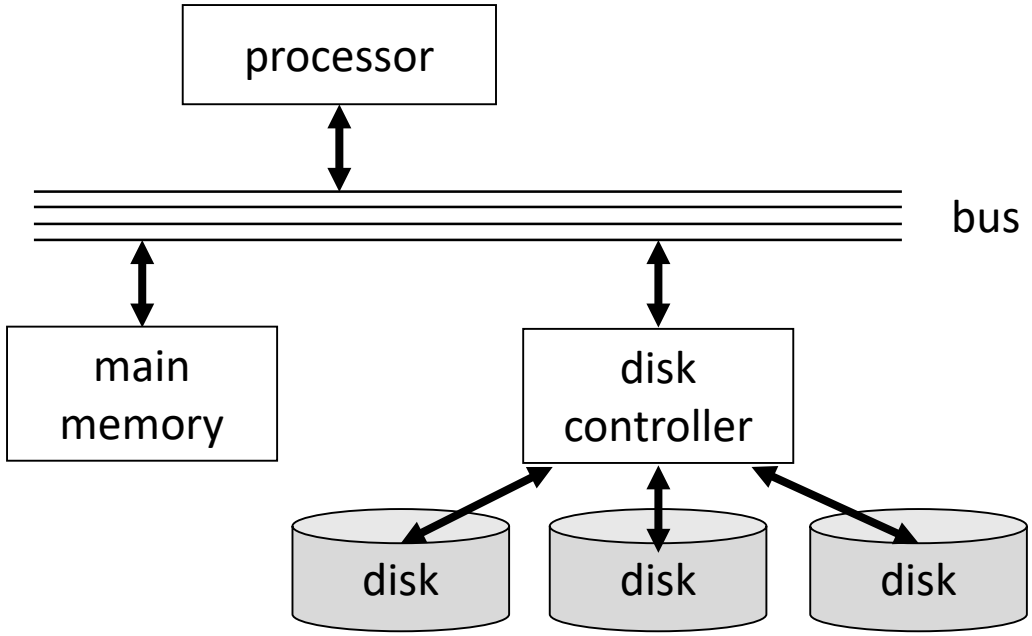


Disks


gap
platter
sector (512 bytes)
track
head
arm movement
rotation
disk arm

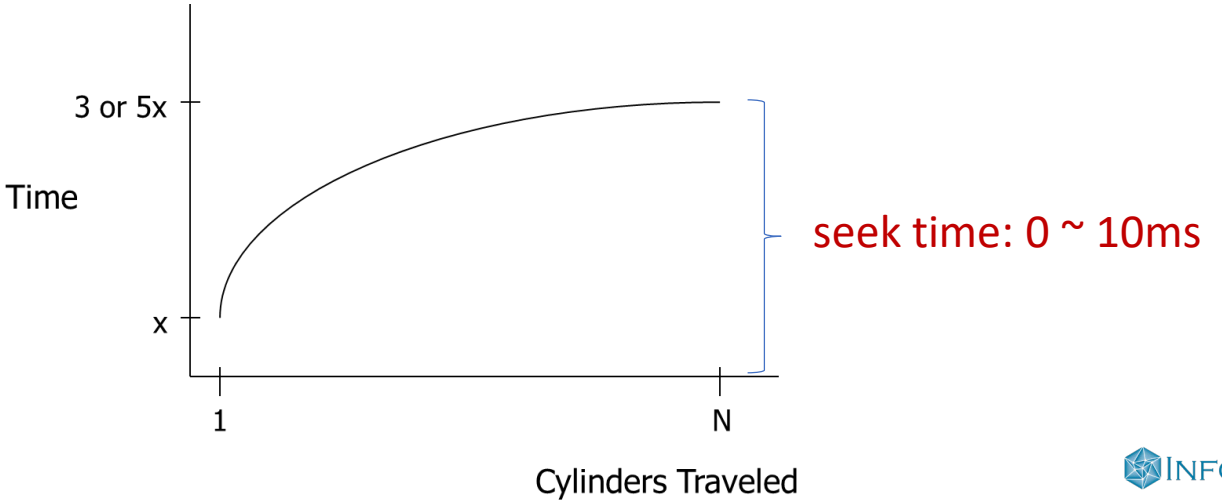
Disk controller

- Control the actuator to position the heads at a track
- Select a sector to read in the track
- Transfer bits between the sector and main memory
- Buffer the track in the local memory of disk controller



Disk access time

- Seek time
  - positioning the head at the track on which the block is located
- Rotational latency
  - waiting while the first sector of the block moves under the head (4.17 ms on average at 7200 RPM)
- Transfer time
  - all the sectors' passing under the head while the disk controller reads (or writes) data in the sectors



Transfer rate: t

- Transfer time = $\frac{size\ of\ blocks}{t}$

- HDD (single): up to 200 MByte/sec
  - transfer time for 4K block : 0.02 ms

- SSD (single): up to 5~6GByte/sec
  - limited by the bandwidth of interface
  - e.g., SATA3: 6Gbit/sec
  - e.g., PCIe 3.0 x4: 4GByte/sec
  - e.g., PCIe 3.0 x8: 8GByte/sec
  - e.g., PCIe 4.0 x4: 8GByte/sec

Rule of thumb

- Random I/O : expensive
- Sequential I/O : much less than random I/O
- e.g., reading 1GB data, block size: 4KB (250,000 blocks), seek time: 4ms, rotational latency: 4ms, transfer rate: 200MB/sec (0.02ms / block)
  - random I/O : 250,000 x (4 + 4 + 0.02) = 2,005,000 ms = 2005 sec
  - sequential I/O : 4 + 4 + 250,000 x 0.02 = 5008 ms = 5 sec
- Similar for SSD

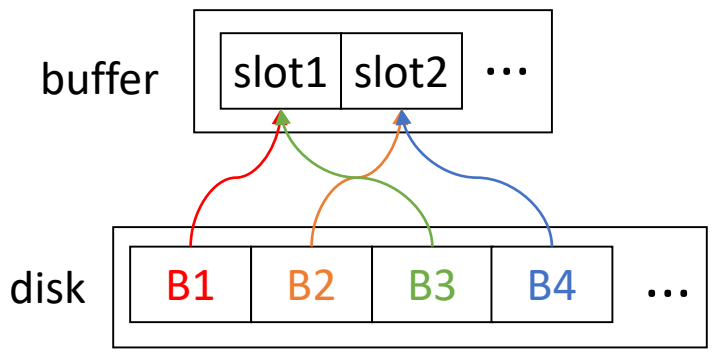
Single buffering technique

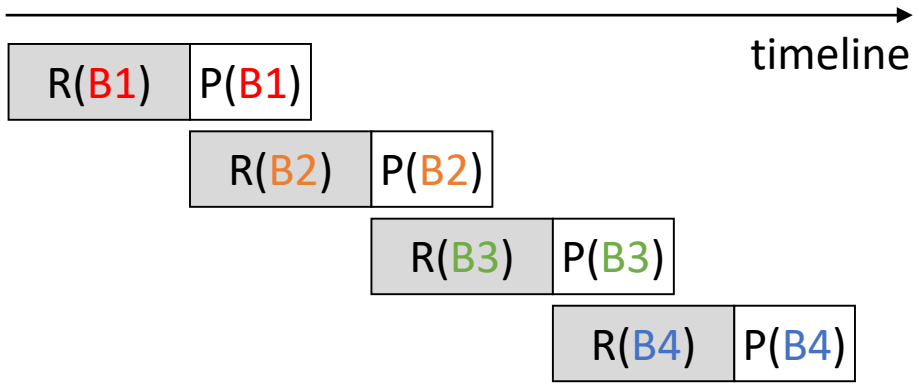
- Reading data from disk and processing data in buffer
- Two approaches
  - reading and processing in separate steps
  - reading and processing in an intertwined manner
- Single buffering
  - read B1 → buffer
  - process data in buffer
  - read B2 → buffer
  - process data in buffer
  - …

sequential

Double buffering technique

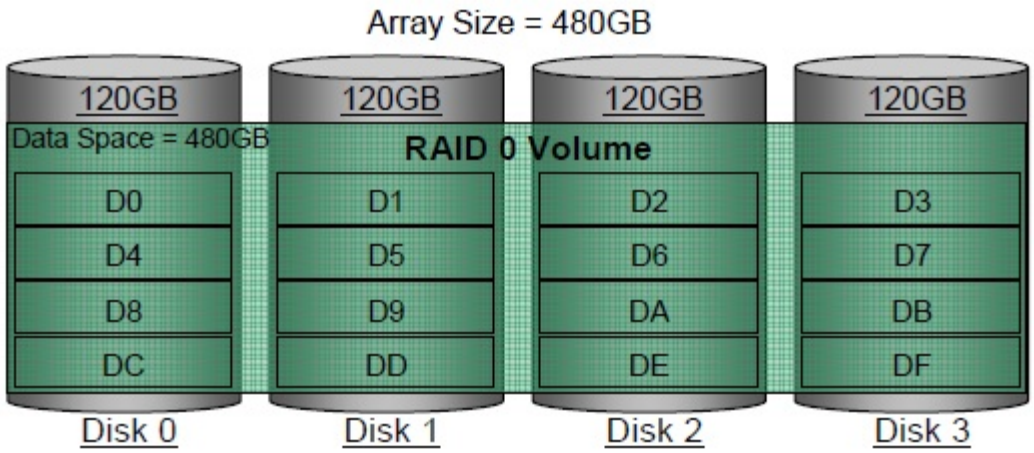
- Notation
  - R(·) : time to read one block to buffer
  - P(·) : time to process one block in buffer
  - n : # of blocks
- Elapsed times
  - R(·) > P(·) : P(·) + nR(·) *data-intensive*
  - R(·) < P(·) : R(·) + nP(·) *compute-intensive*



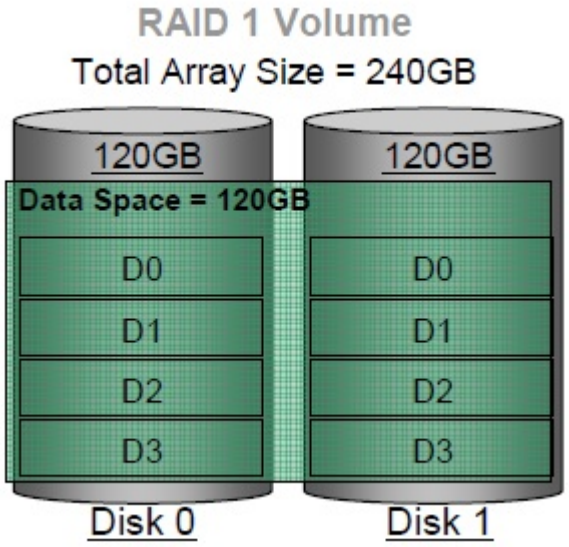


RAID 0 vs. RAID 1

RAID 0
Fast (>x2)
But, not fault-tolerant
(min. # disks = 2)

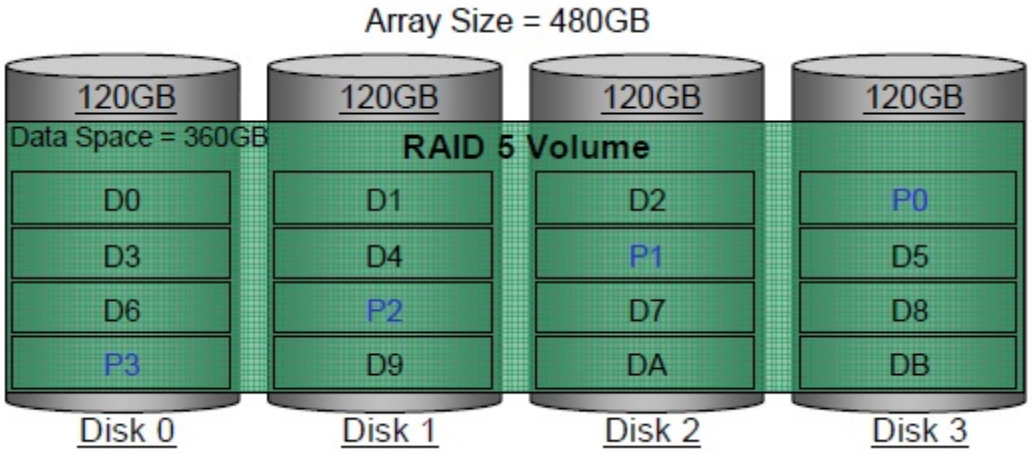


RAID 1
Fault-tolerant
But, large disk space (x2)
(min. # disks = 2)

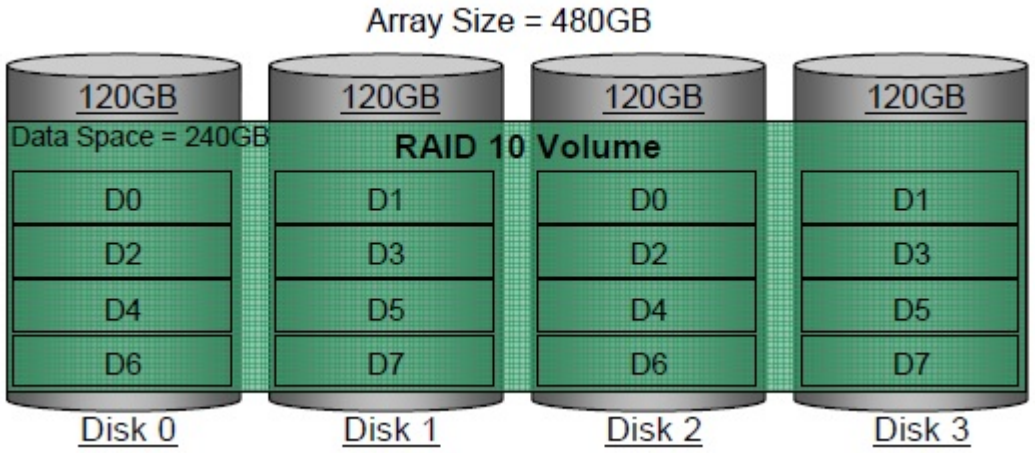


RAID 5 vs. RAID 10

RAID 5
All-round
(min. # disks = 3)

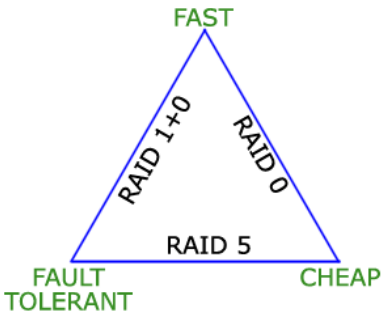


RAID 10
Combination of
RAID 1 + RAID 0
(min. # disks = 4)



Comparison of Common RAID types

RAID	Min Disks	Storage Efficiency %	Cost	Read Performance	Write Performance	Write* Penalty	Protection
0	2	100	Low	Good for both random and sequential reads	Excellent	1	No Protection
1	2	50	High	Better than a single disk	Good - Slower than a single disk because every write must be committed to all disks	2	Mirror Protection
5	3	67 - 97%	Moderate - Low	Good for random and sequential reads	Fair for random and sequential writes	4	Parity protection for single disk failure
6	3	33 - 94%	Extremely High - Low	Good for random and sequential reads	Poor to fair for random writes and fair for sequential writes	6	Parity protection for two disk failures
10	4	50	High	Good	Good	2	Mirror Protection
50	6	67 - 94%	Moderate-High	Better	Moderate for random and sequential writes	4	Parity protection for single disk failure
60	6	33 - 88%	Extremely High - Moderate	Better	Fair for random and sequential writes	6	Parity protection for two disk failures



Arranging data on disk

- Tuple or object is represented by a record
- Record consists of consecutive bytes in disk block (in row-oriented layout)
- Records are eventually manipulated in main memory
  - although they are kept in secondary storage
  - necessary to lay out the records so it can be moved to main memory and accessed efficiently there

Structure of a record

- Record begins with header
  - header: a fixed-length information about the record itself
- Header contains
  - pointer to the schema for the data stored in the record
  - length of the record
    - help us skip over records without consulting the schema
  - timestamps indicating the time the record was last modified, or last read
    - useful for implementing database transactions
  - pointers to the fields of the record
    - can substitute for schema information

Example of a fixed-length record

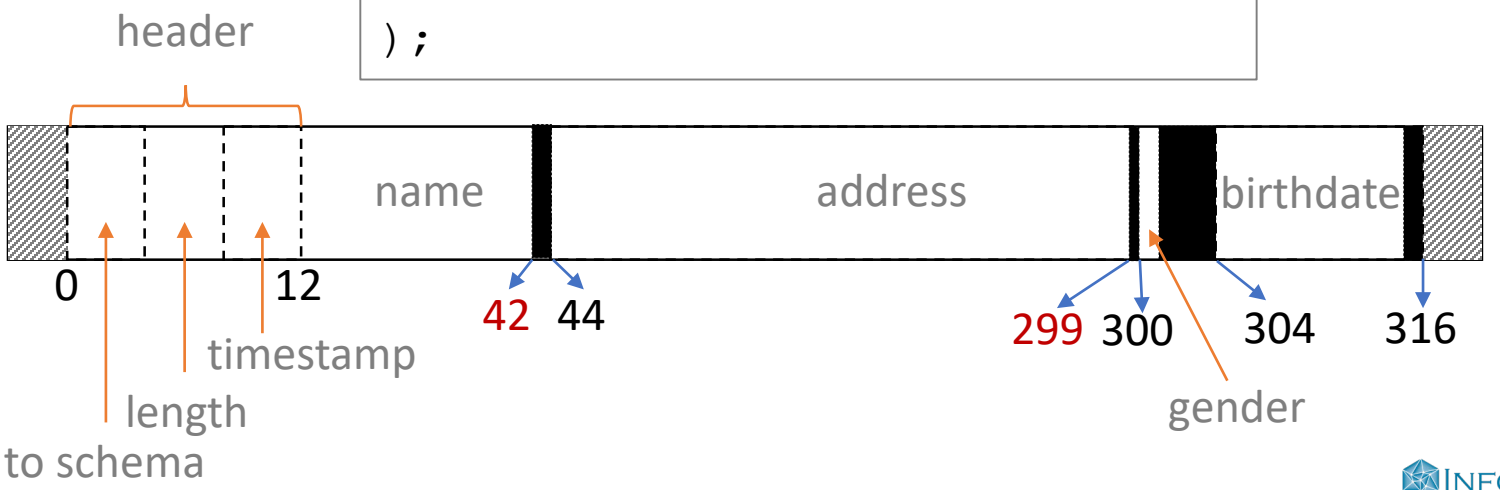
- Assumptions
  - all fields must start at a byte that is a multiple of four (memory alignment)
  - DATE type is a 10-byte string

```
CREATE TABLE MovieStar (
    name CHAR(30) PRIMARY KEY,
    address VARCHAR(255),
    gender CHAR(1),
    birthdate DATE
);
```

→ 10 bytes

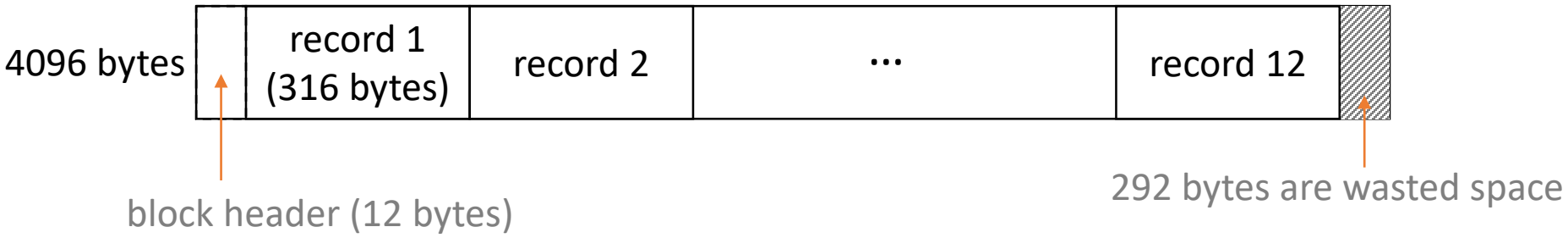
not 296 bytes long!

316 bytes long



Packing fixed-length records into blocks

- Records of a relation are stored in blocks of the disk
- Block header holds
  - links to one or more other blocks (part of a network of blocks)
  - information about the role of this block in the network
  - information about which relation this block belongs to
    - a block usually holds tuples from one relation
  - directory giving the offset of each record in the block
  - timestamp(s): time of the last modification and/or access



Representing block and record addresses

- In main memory
  - address of a block: virtual-memory address of its first byte
  - address of a record: virtual-memory address of its first byte

- In secondary storage
  - block is NOT part of the application's virtual-memory address space
  - address of a block: device ID for the disk, cylinder number, track, …
  - address of a record: block address + offset of its first byte within the block

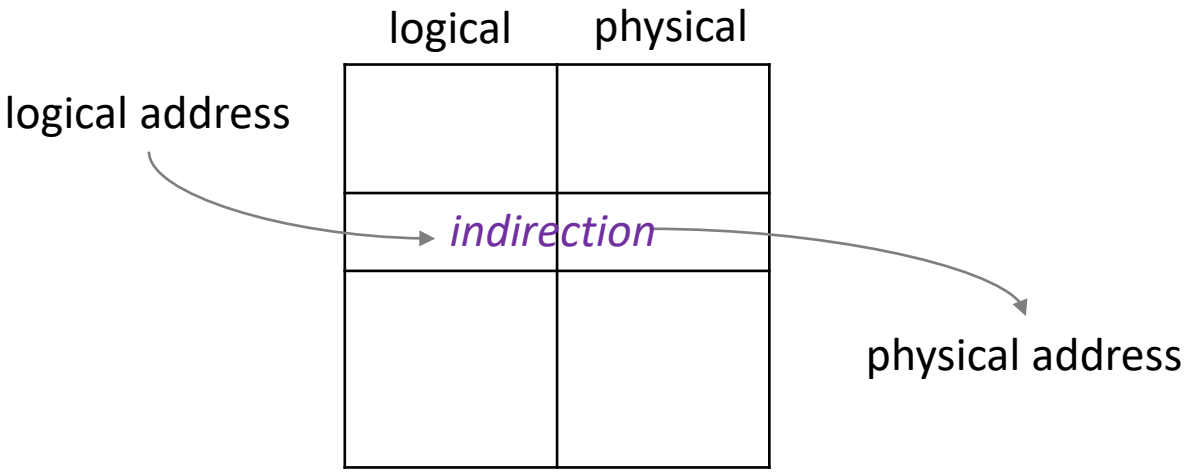
Database address space

- All data objects in DBMS lives in this address space
  - data objects: blocks, records, ..

- Physical addresses
  - host to which the storage is attached
  - identifier for the disk on which the block is located
  - number of the cylinder
  - number of the track within the cylinder
  - number of the block within the track
  - offset of the beginning of the record within the block

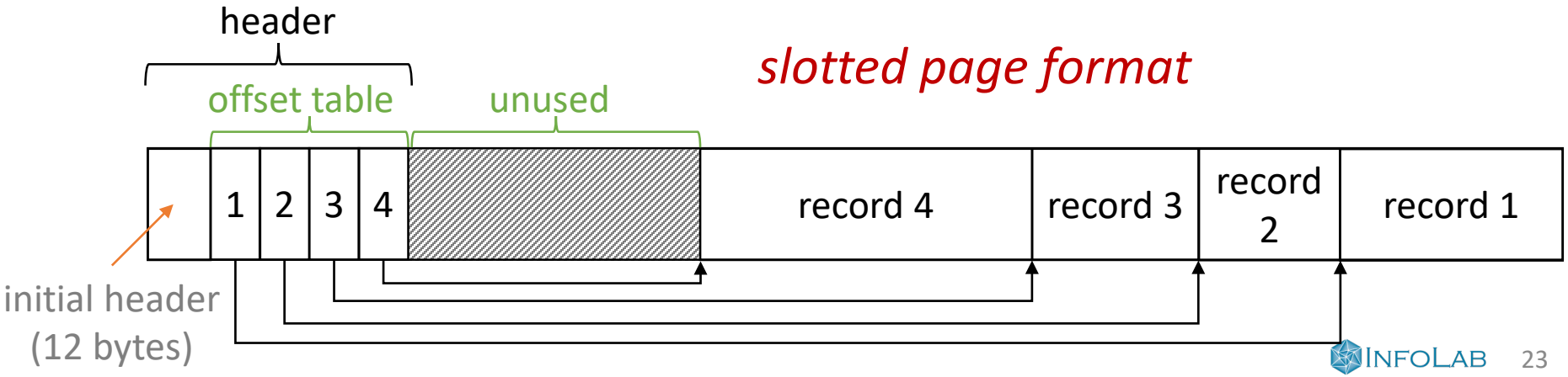
Logical addresses

- Each block or record has a logical address
  - arbitrary string of bytes of some fixed length
- (Global) Map table
  - stored on disk in a known location
  - translate logical to physical addresses
  - logical address is shorter than physical address



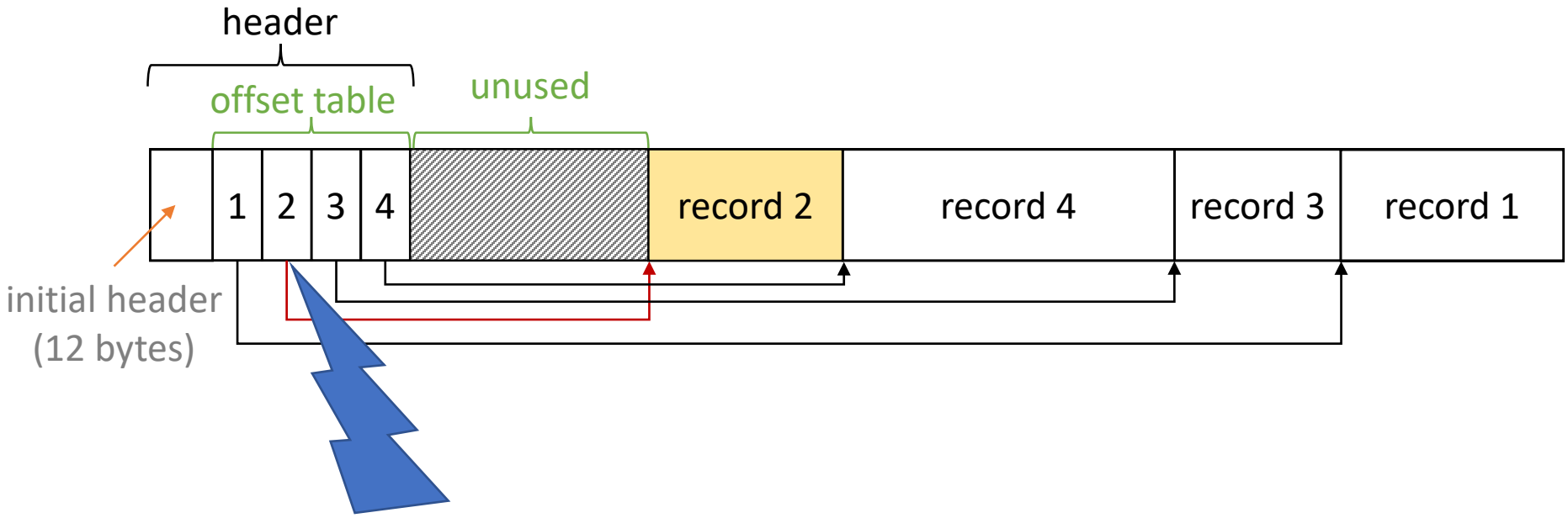
Logical and structured addresses

- Flexibility of indirection using the global map table
  - a record can be referenced by a lot of other records
  - it is often to move records within a block or from block to block
  - all pointers to the record refer the map table
  - just change the table entry when moving or deleting the record
- Combination of logical and physical addresses
  - use a physical address for the block
  - keep an offset table for the records in each block



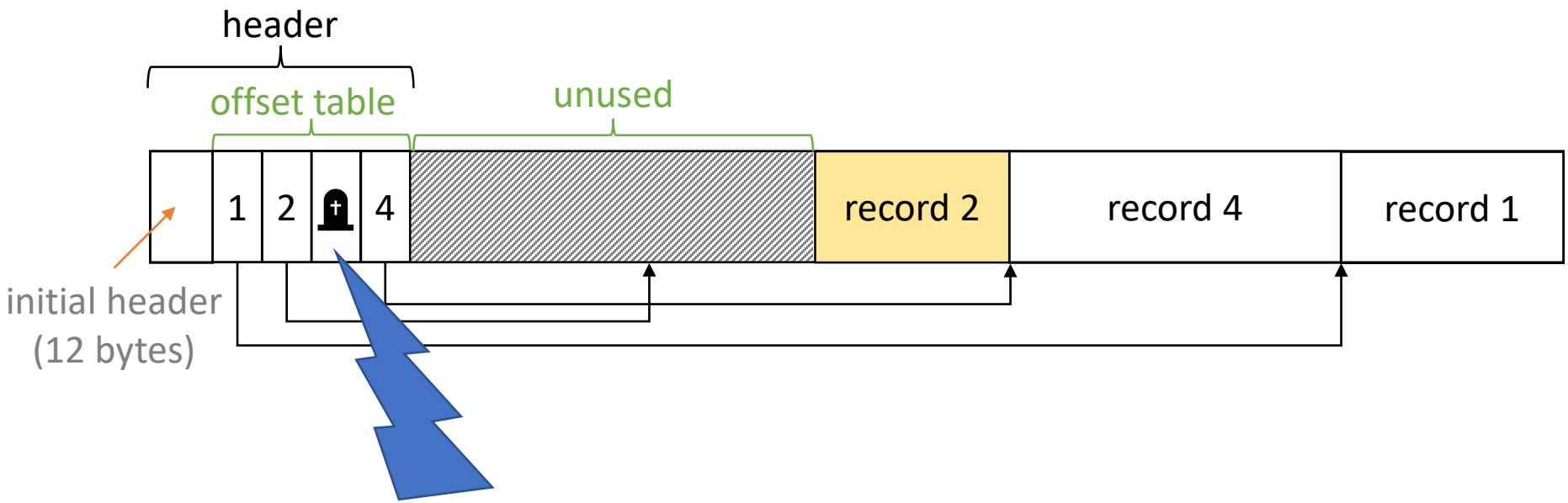
Advantages of indirection within the block

- We can move the record around within the block
  - all we have to do is change the record's entry in the offset table
  - pointers to the record will still be able to find it



- We can even allow the record to move to another block
  - if the offset table entries are large enough to hold a forwarding address for the record, giving its new location

- We can handle the deletion of the record easily
  - leave a tombstone in the record's entry in the offset table
  - solve a kind of dangling pointer problem



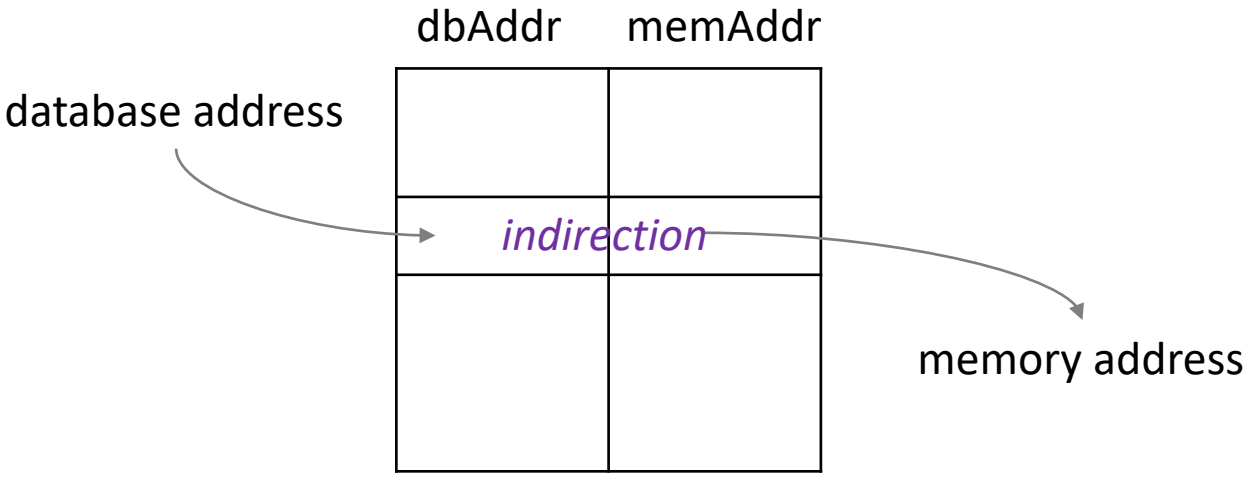
Management of pointers

- Database address
  - both physical address and logical address are database address
  - they are not memory address in current virtual memory
  - we must use database address in secondary storage

- Memory address
  - we can refer to the data item (e.g., block, record, object) by either its database address or its memory address
  - we need a translation table if we use database address
  - memory address is much more efficient (in single instruction)

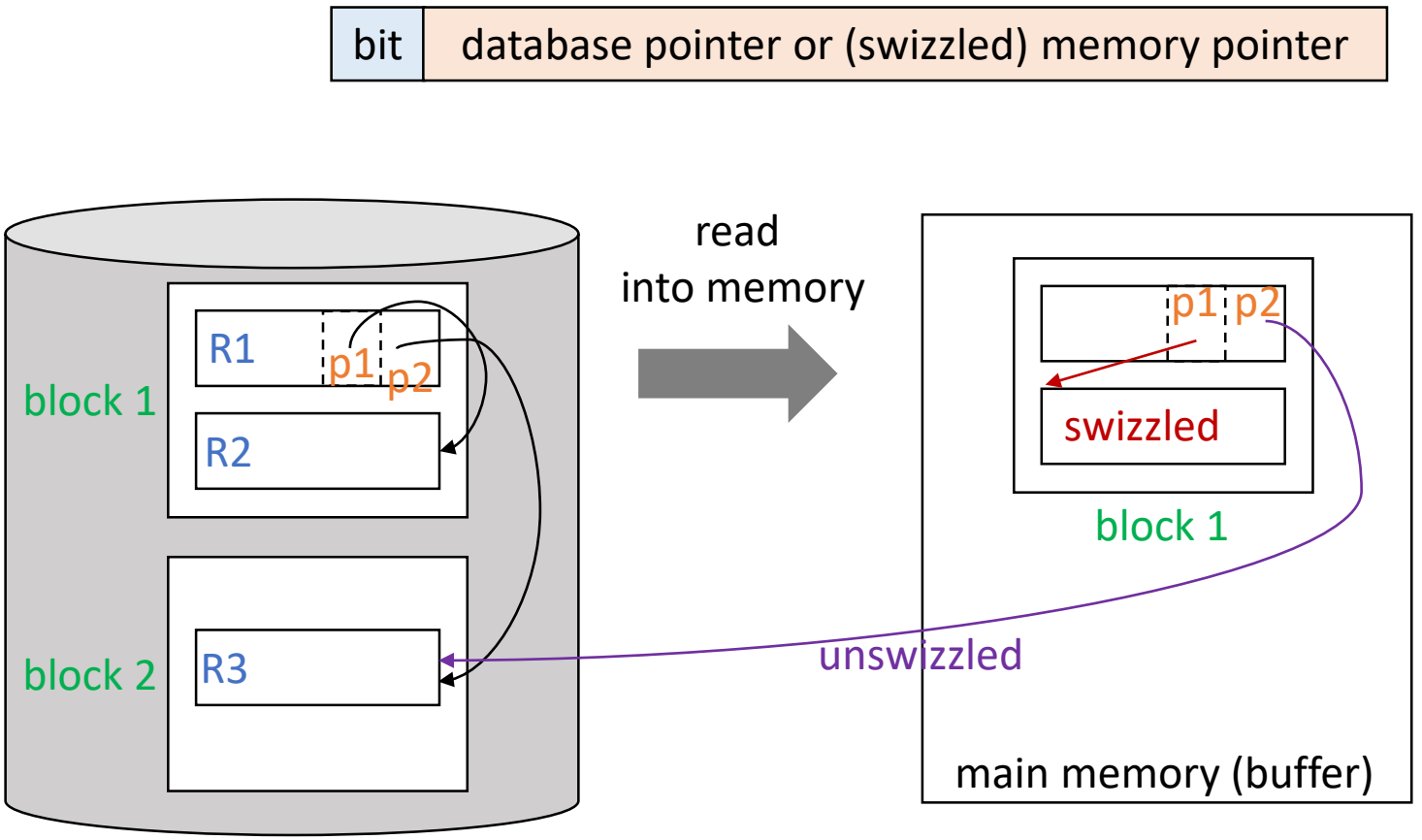
Translation table

- Translate from database address to memory address
  - only the items currently in virtual memory have entries in the translation table
    - c.f., all addressable items in the database have entries in the map table
  - memory addresses in the translation table are for copies of the corresponding object in memory
  - time consuming

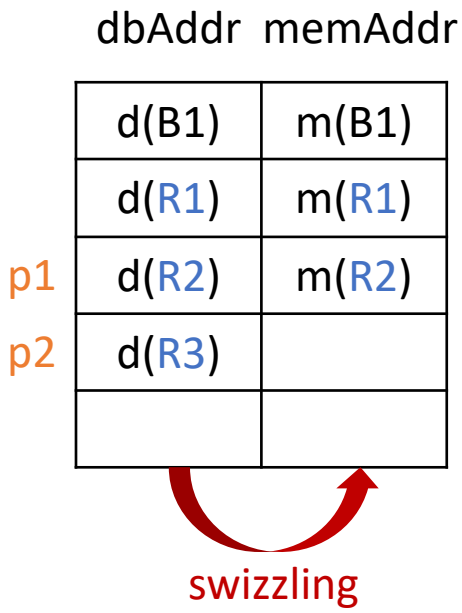


Pointer swizzling

- Techniques for avoiding the cost of translating repeatedly from DB address to Mem address
  - we "swizzle" the pointers within the block when moving a block from secondary storage to main memory



Automatic swizzling



- As soon as a block is brought into memory
  - locate all its pointers and addresses
    - pointers from records in the block to elsewhere
    - address of the block itself
    - addresses of the records in the block
  - enter them into the translation table (if they are not there)

- Swizzling pointers
  - case 1: finding the memory address for a pointer in the table
    - replacing the database address by the memory address
    - set the swizzled bit to true
  - case 2: finding a pointer is still unswizzled and not in the table
    - load the corresponding block into memory buffer
    - swizzle the pointer

Swizzling on demand

- As soon as a block is brought into memory
  - locate all its addresses and enter them into the translation table (if they are not there)
  - but, leave all its pointers unswizzled
- Comparison with automatic swizzling
  - automatic swizzling: all the pointers are swizzled quickly and efficiently when the block is loaded into memory
  - on-demand swizzling: not waste the time spent for swizzling the pointers that won't be used (followed) eventually
- c.f., no swizzling
  - need the translation table
  - offer advantage that the records cannot be pinned in memory

Returning blocks to disk

- When a block is moved from memory back to disk
  - any pointers within the block must be unswizzled
  - memory addresses of the pointers must be replaced by the corresponding database addresses

- If we think of the translation table as a relation

```
SELECT memAddr
FROM TranslationTable
WHERE dbAddr = x;
```

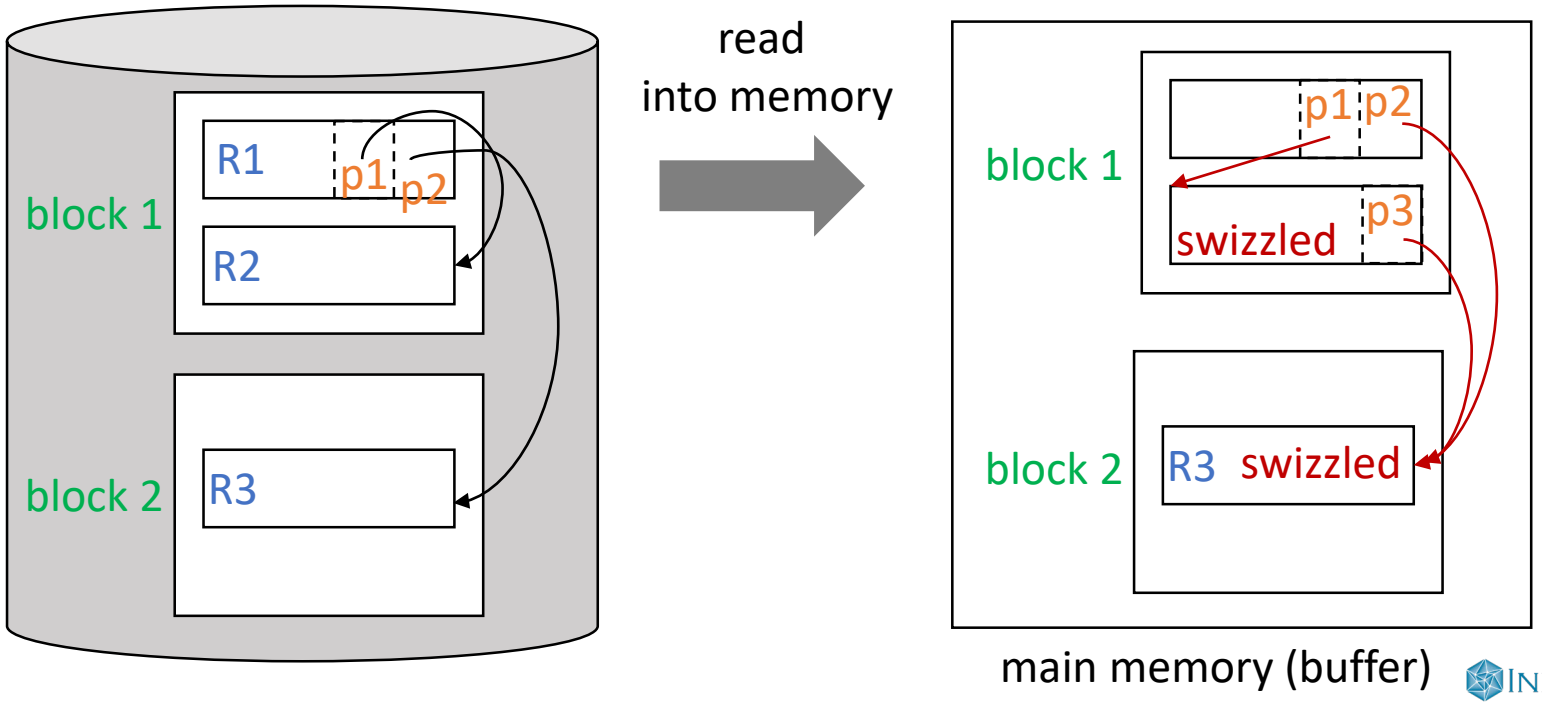
for swizzling

```
SELECT dbAddr
FROM TranslationTable
WHERE memAddr = y;
```

for unswizzling

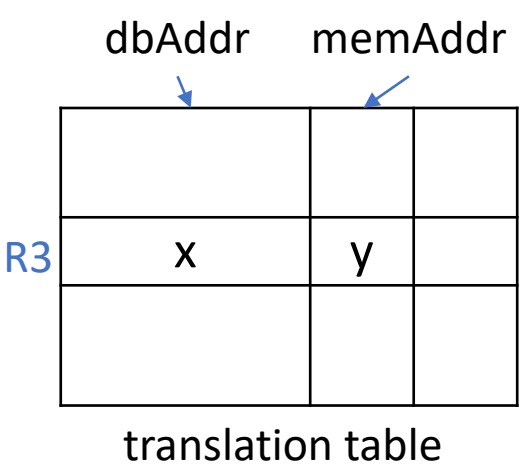
Pinned records and blocks (1)

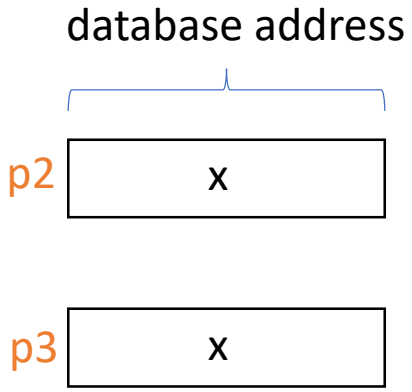
- Pinned block: cannot be written back to disk safely
  - e.g., B_1 has a swizzled pointer to data item in B_2, moving B_2 back to disk and reusing its main memory buffer makes the pointer dangling (B_2 is therefore pinned)



Pinned records and blocks (2)

- To write the block back to disk
  - unswizzle any pointers in the block
  - unpin the block by unswizzling any pointers to it (e.g., p2)
- To unpin the block, use a linked list
  - memory address is significantly shorter than database address

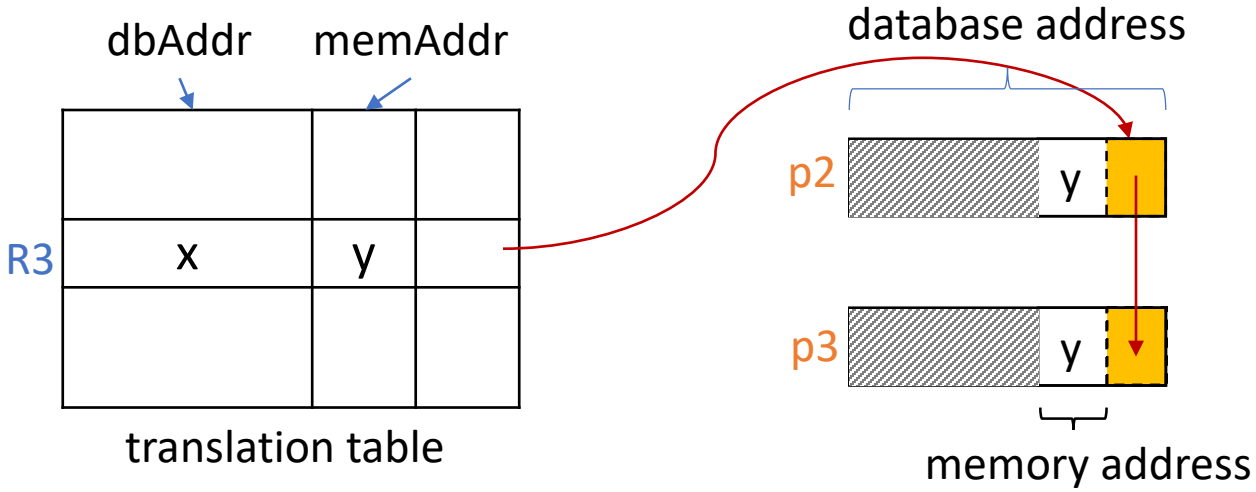




before p2 and p3 are swizzled

Pinned records and blocks (2)

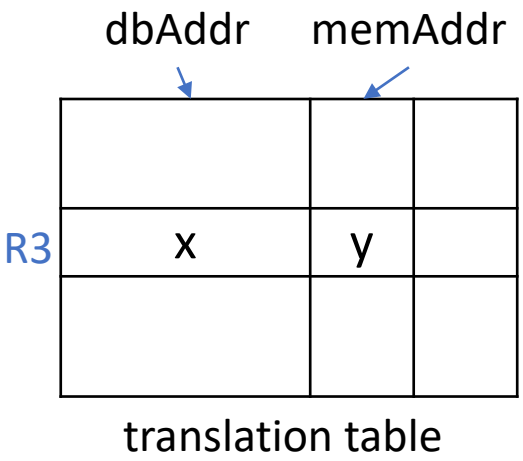
- To write the block back to disk
  - unswizzle any pointers in the block
  - unpin the block by unswizzling any pointers to it (e.g., p2)
- To unpin the block, use a linked list
  - memory address is significantly shorter than database address

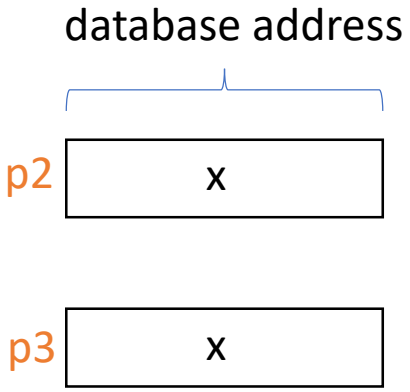


after p2 and p3 are swizzled

Pinned records and blocks (2)

- To write the block back to disk
  - unswizzle any pointers in the block
  - unpin the block by unswizzling any pointers to it (e.g., p2)
- To unpin the block, use a linked list
  - memory address is significantly shorter than database address





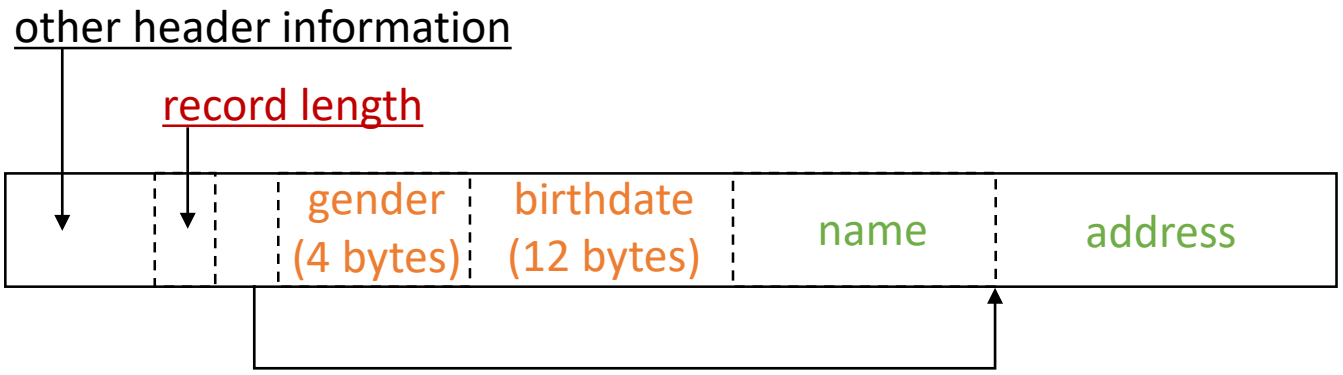
after p2 and p3 are **unswizzled**

Variable-length data and records

- Data items whose size varies (e.g., address)
- Repeating fields
  - we need to store references to as many objects as are related to the given object, if there is M:N relationship
- Variable-format records
  - sometimes we do not know in advance what the fields of a record will be (e.g., XML, JSON)
- Enormous fields
  - sometimes attribute values are very large (e.g., audio, image, video)

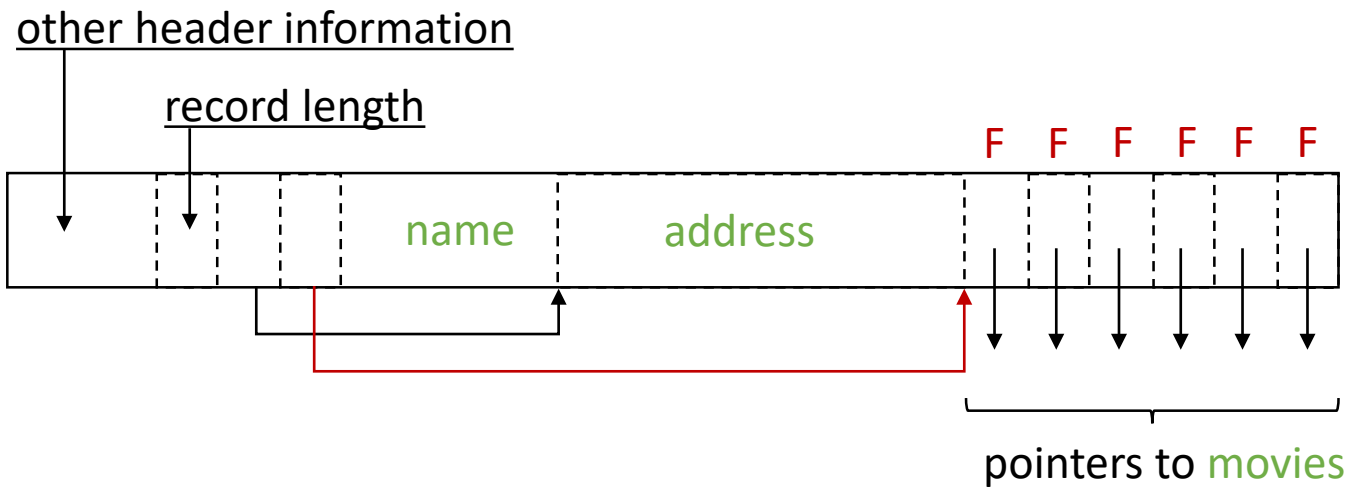
Records with variable-length fields

- Put all fixed-length fields ahead of the variable-length fields
- Record header
  - length of the record
  - pointers to the beginnings of all the variable-length fields other than the first one



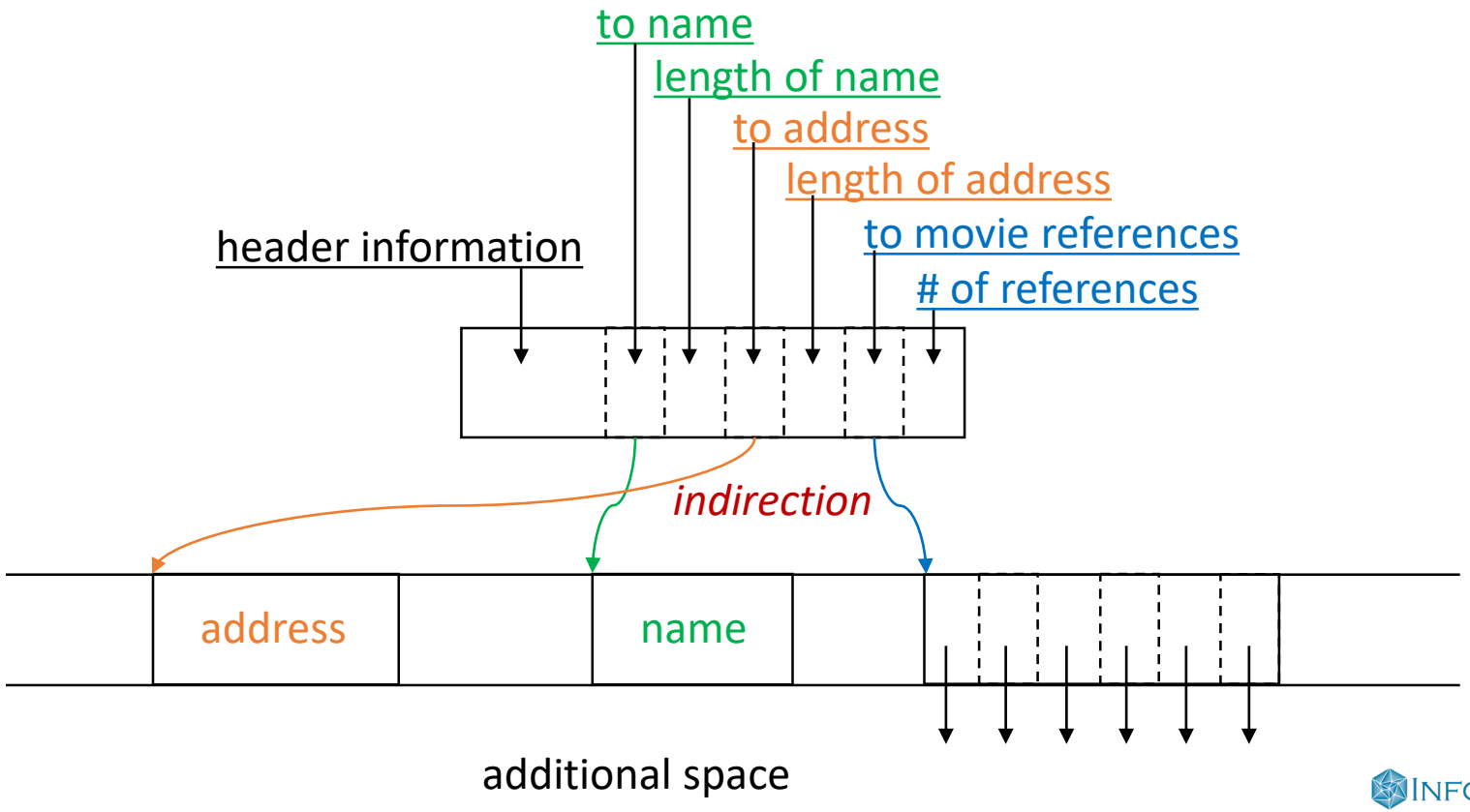
Records with repeating fields

- Assumption
  - a record contains a variable number of occurrences of a field F
  - field itself is of fixed length (L: number of bytes for a field)



Records with repeating fields (alternative)

- Put the variable-length portion on a separate block
  - records can be searched or moved more efficiently
  - but, the number of disk I/O's increases



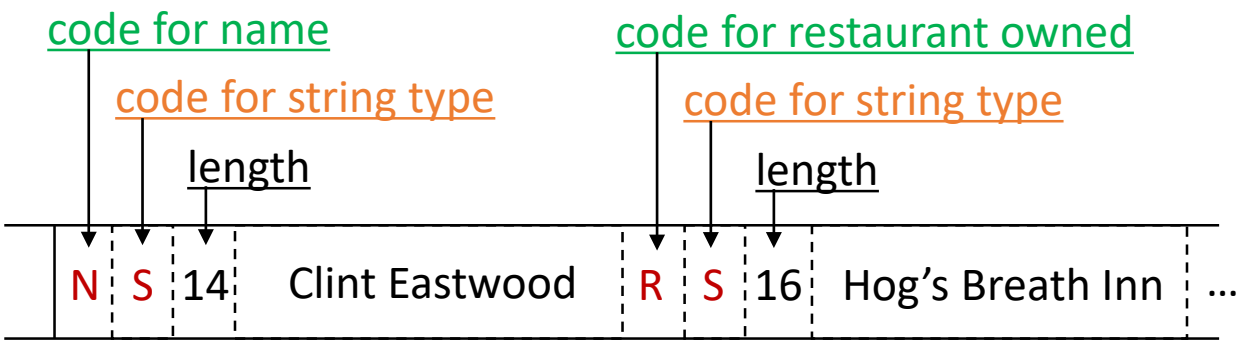
Variable-format records

```
<Star>
    <name> Clint Eastwood
    </name>
    <restaurant owned>
        Hog's Breath Inn
    </restaurant owned>
</Star>
```

- e.g., XML, medical records
- Sequence of tagged fields
  - value of the field
  - information about the role of the field
    - attribute or field name
    - type of the field
    - length of the field

code table

code	value
N	"name"
R	"restaurant owned"
S	string type



movie stars may have additional attributes such as restaurants owned

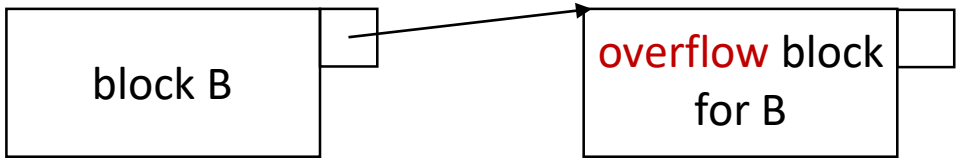
Binary, Large OBjects (BLOBs)

- Storage of BLOBs
  - stored on a sequence of blocks (spanned record)
  - it is preferred that blocks are allocated consecutively on disk
  - it is necessary to stripe BLOBs across disks for faster retrieval

- Retrieval of BLOBs (in client-server environment)
  - pass only the small fields of the record
  - allow the client to request blocks of the BLOB one at a time
  - client should be able to request interior portions of the BLOB without having to receive the entire BLOB
  - it requires a suitable index structure (e.g., an index by seconds on a movie BLOB)

Record modification - insertion

- Problem: when tuples must be kept in some fixed order
- Case 1: there is room in the block → slotted page format
- Case 2: there is no room in the block for the new record
  - find space on a nearby block
    - look at the following block in the sorted order
    - slide the records around on both blocks
  - create an overflow block
    - each block B has in its header a place for a pointer to an overflow block
    - additional records in the overflow blocks theoretically belong in B



Record modification - deletion

- Reclaiming the space after deleting a record
  - slide records
  - maintain an available-space list (like garbage collection)
    - put the list head in the block header
    - each available region holds the link to the next region
- Not able to reclaim due to pointers to the delete record
  - place a tombstone in place of the deleted record
    - tombstone is permanent; must exist until the entire DB is reconstructed
  - tombstone could be a null pointer
    - in the offset table if we use the slotted page format
    - in the physical address entry if we use the map table

Thank you

Q & A